

Prática DevOps: Kubernetes Social Ledger

Disciplina: DevOps

Autor: João Paulo Morais Rangel

1. Visão Geral da Implantação

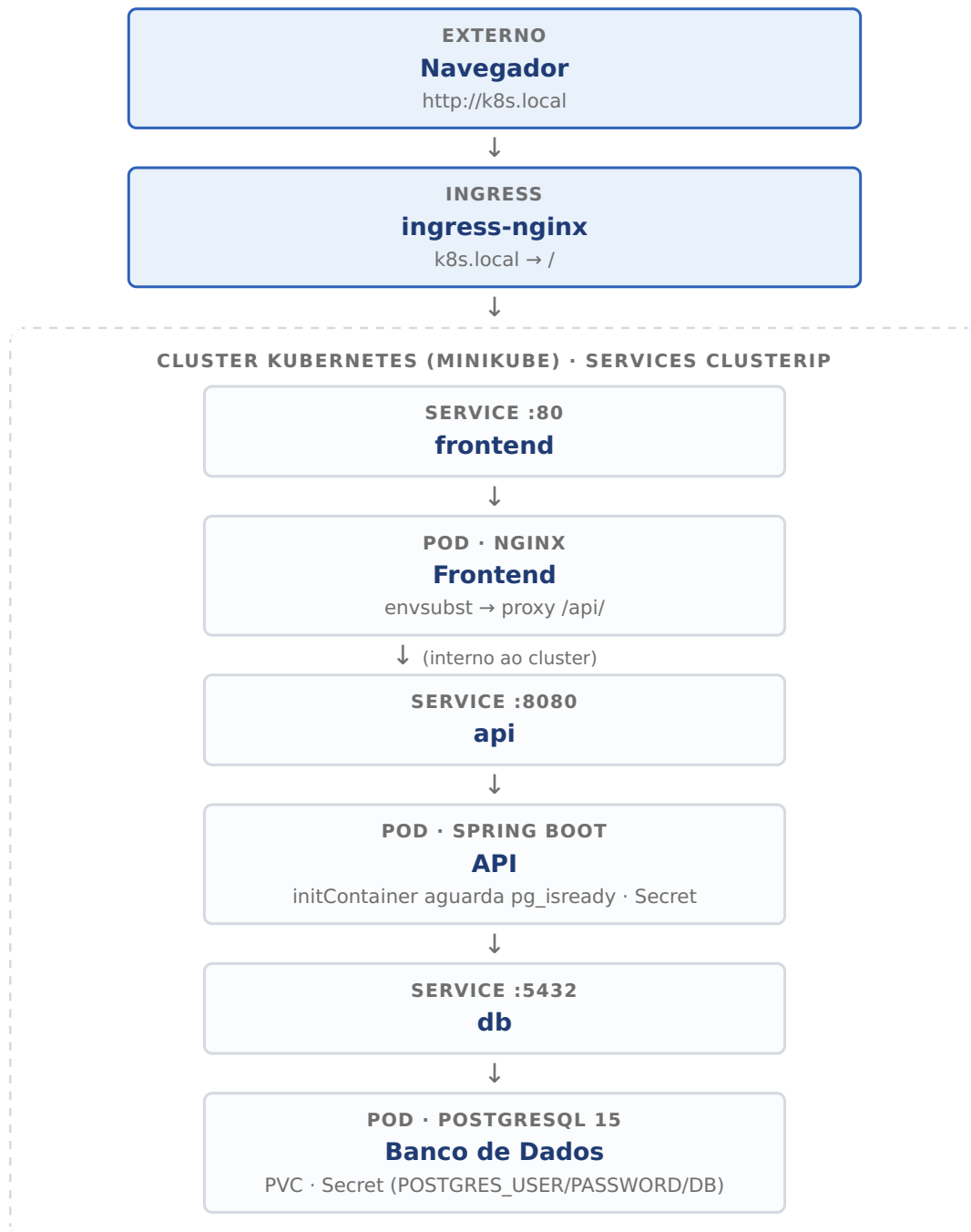
O **Social Ledger** é uma ferramenta full-stack de gestão de despesas compartilhadas, já descrita e containerizada na entrega anterior. Nesta etapa, a mesma aplicação é migrada do Docker Compose para um cluster **Kubernetes local (Minikube)**, utilizando **Helm** como gerenciador de pacotes para automatizar e parametrizar toda a implantação.

A aplicação continua composta pelos três componentes originais — banco de dados PostgreSQL, API Spring Boot e frontend React/Nginx — agora representados como objetos nativos do Kubernetes: Deployments, Services, Secret, PersistentVolumeClaim e Ingress.

Camada	Tecnologia	Descrição / Papel
Backend	Java 21 / Spring Boot 4	API RESTful com arquitetura limpa.
Frontend	React / TypeScript / Vite	SPA servida pelo Nginx como proxy reverso.
Banco de Dados	PostgreSQL 15	Persistência relacional com volume persistente.
Orquestração	Kubernetes / Minikube	Cluster local para implantação dos pods.
Empacotamento	Helm 3	Chart parametrizável para instalar/atualizar toda a aplicação com um único comando.
Roteamento	Nginx Ingress Controller	Exposição da aplicação em <code>http://k8s.local</code> .

2. Arquitetura no Kubernetes

Todos os recursos são implantados no namespace `default` do Minikube. A comunicação entre os pods utiliza exclusivamente DNS interno do cluster, via os Services do tipo **ClusterIP**. O único ponto de entrada externo é o **Ingress**, que roteia o tráfego de `k8s.local` para o Service do frontend. A API e o banco de dados não possuem exposição externa.



2.1. Pod de Banco de Dados (social-ledger-db)

- **Imagem:** `postgres:15-alpine`.
- **Persistência:** PersistentVolumeClaim de 1 Gi montado em `/var/lib/postgresql/data`. Os dados sobrevivem a reinicializações e recriações do pod.
- **Credenciais:** as variáveis `POSTGRES_DB`, `POSTGRES_USER` e `POSTGRES_PASSWORD` são injetadas a partir do Secret, nunca hardcoded no manifest.
- **Health checks:** `readinessProbe` e `livenessProbe` executam `pg_isready`, equivalente ao `healthcheck` do Docker Compose.

2.2. Pod de API (social-ledger-api)

- **Imagem:** construída com Multi-stage build (Maven + JRE Alpine) e publicada no Docker Hub.

- **Dependência:** um `initContainer` bloqueia a inicialização do pod até o PostgreSQL responder ao `pg_isready` — equivalente ao `depends_on: condition: service_healthy` do Compose.
- **Configuração:** as variáveis `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` e `SPRING_DATASOURCE_PASSWORD` são injetadas a partir do Secret via `secretKeyRef`.
- **Health checks:** `readinessProbe` e `livenessProbe` via `tcpSocket` na porta 8080.

2.3. Pod de Frontend (social-ledger-frontend)

- **Imagem:** construída com Multi-stage build (Node/Vite + Nginx Alpine) e publicada no Docker Hub.
- **Configuração via envsubst:** a imagem oficial do Nginx processa na inicialização um template em `/etc/nginx/templates/`, substituindo a variável de ambiente `API_HOST` pelo nome do Service. Isso configura o proxy reverso de `/api/` para `social-ledger-api:8080` via DNS interno, sem necessidade de ConfigMap e eliminando problemas de CORS no navegador.
- **Health checks:** `readinessProbe` e `livenessProbe` via `httpGet /`.

3. Artefatos Kubernetes

Todos os artefatos abaixo são templates Helm localizados em `helm/social-ledger/templates/`. Os nomes dos recursos são prefixados com o nome da release Helm (padrão: `social-ledger`), tornando possível instalar múltiplas instâncias no mesmo cluster.

3.1. Secret — `secret.yaml`

Armazena as credenciais do banco de dados (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`) codificadas em Base64 com o tipo `Opaque`. Os Deployments do banco e da API referenciam esses valores via `secretKeyRef`, garantindo que senhas nunca apareçam em texto claro nos manifests dos pods.

```
apiVersion: v1
kind: Secret
metadata:
  name: social-ledger-db-secret
type: Opaque
data:
  postgres-db:      "c29jaWFsbGVkZ2Vy"    # socialledger (base64)
  postgres-user:    "cG9zdGdyZXM="      # postgres (base64)
  postgres-password: "cG9zdGdyZXM="      # postgres (base64)
```

3.2. PersistentVolumeClaim — `db/pvc.yaml`

Solicita ao cluster um volume persistente de **1 Gi** com modo de acesso `ReadWriteOnce` (leitura/escrita por um único nó). O volume é montado no caminho de dados do PostgreSQL. Mesmo que o pod seja recriado ou o Deployment atualizado, os dados do banco são preservados enquanto o PVC existir.

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: social-ledger-db-pvc
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi

```

3.3. Deployments

Um **Deployment** é o objeto responsável por declarar o estado desejado de um conjunto de pods — quantas réplicas rodar, qual imagem usar, variáveis de ambiente, volumes e sondas de saúde. O Kubernetes garante continuamente que o estado real do cluster corresponda ao estado declarado.

Deployment	Imagem	Recurso especial
social-ledger-db	postgres:15-alpine	Monta o PVC; credenciais via Secret.
social-ledger-api	<user>/social-ledger-api:latest	initContainer aguarda banco; env vars via Secret.
social-ledger-frontend	<user>/social-ledger-frontend:latest	Recebe API_HOST via env; Nginx configurado por envsubst.

Todos os Deployments possuem `readinessProbe` (o pod só recebe tráfego quando pronto) e `livenessProbe` (o pod é reiniciado automaticamente se travar), além de `resources.requests` e `resources.limits` para controle de CPU e memória.

3.4. Services

Um **Service** expõe um conjunto de pods sob um endereço DNS estável dentro do cluster. Todos os três Services são do tipo **ClusterIP** — acessíveis apenas internamente. O Service do frontend é o único referenciado pelo Ingress.

Service	Porta	Consumidor
social-ledger-db	5432	API (via <code>SPRING_DATASOURCE_URL</code>)
social-ledger-api	8080	Frontend (nginx <code>proxy_pass</code>)
social-ledger-frontend	80	Ingress

3.5. Configuração do Nginx via envsubst

Em vez de um ConfigMap montado como volume, a configuração do Nginx é resolvida pela própria imagem. A imagem oficial `nginx:alpine` processa automaticamente, na inicialização, qualquer arquivo com extensão `.template` em `/etc/nginx/templates/`: ela substitui as variáveis de ambiente e grava o resultado em `/etc/nginx/conf.d/`. Isso elimina um artefato Kubernetes e torna o container autocontido.

O template usa `${API_HOST}` no `proxy_pass`, e o valor é injetado por variável de ambiente — `social-ledger-api` no Kubernetes e `api` no Docker Compose. A mesma imagem serve os dois ambientes.

```
# nginx.conf (template)
location /api/ {
    proxy_pass http://${API_HOST}:8080/api/;
    proxy_set_header Host      $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# Dockerfile
COPY nginx.conf /etc/nginx/templates/default.conf.template

# Deployment (frontend)
env:
  - name: API_HOST
    value: "social-ledger-api"
```

3.6. Ingress — `ingress.yaml`

O **Ingress** é o único ponto de entrada externo da aplicação. Utilizando o controlador `nginx` habilitado como addon do Minikube, roteia todo tráfego recebido no host `k8s.local` para o Service do frontend na porta 80. A API **não é exposta diretamente** pelo Ingress — permanece acessível apenas internamente via o proxy reverso do Nginx no pod do frontend.

```
spec:
  ingressClassName: nginx
  rules:
    - host: k8s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: social-ledger-frontend
                port:
                  number: 80
```

| 4. Helm Chart

O **Helm** é o gerenciador de pacotes do Kubernetes. Um *chart* é um conjunto de templates parametrizáveis que, ao serem instalados, geram todos os objetos Kubernetes necessários para a aplicação. Os valores configuráveis (imagens, credenciais, recursos) são centralizados em `values.yaml`.

Estrutura do Chart

```

helm/social-ledger/
├── Chart.yaml           # metadados (nome, versão, descrição)
├── values.yaml          # valores padrão configuráveis
├── templates/
│   ├── _helpers.tpl     # funções reutilizáveis (nomes, labels)
│   ├── secret.yaml      # Secret compartilhado
│   ├── ingress.yaml     # Ingress global
│   ├── db/
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── pvc.yaml
│   ├── api/
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── frontend/
│       ├── deployment.yaml
│       └── service.yaml

```

Estrutura por componente: os templates são organizados em subdiretórios por serviço (db/, api/, frontend/), facilitando a adição de novos microserviços sem poluir os demais.

5. Manual de Instalação e Execução

Pré-requisitos

- Docker instalado e em execução.
- Minikube instalado (minikube start já executado).
- Helm 3 instalado.
- Conta no Docker Hub com as imagens publicadas.

Passo 1 — Configurar o usuário Docker Hub

Abra o arquivo helm/social-ledger/values.yaml e substitua jpmrangel pelo seu usuário do Docker Hub nos campos api.image.repository e frontend.image.repository.

Passo 2 — Habilitar o addon Ingress no Minikube

```
minikube addons enable ingress
```

Aguarde o controlador do Ingress ficar no estado Running:

```
kubectl get pods -n ingress-nginx -w
```

Passo 3 — Build, push e carga no Minikube

O script build.sh na raiz do projeto automatiza as três etapas: build das imagens Docker, envio ao Docker Hub e carregamento local no Minikube.

```

chmod +x build.sh
./build.sh <seu-usuario-dockerhub> latest

```

Passo 4 — Instalar o Helm Chart

```
helm upgrade --install social-ledger ./helm/social-ledger
```

Aguarde todos os pods ficarem no estado `Running`:

```
kubectl get pods -w
```

Passo 5 — Configurar resolução de nome local

Adicione o IP do Minikube ao arquivo `/etc/hosts`:

```
echo "$(minikube ip) k8s.local" | sudo tee -a /etc/hosts
```

Passo 6 — Acessar a aplicação

Abra o navegador em: **`http://k8s.local`**

6. Roteiro de Testes (Passo a Passo)

Para validar o funcionamento completo da aplicação no cluster Kubernetes, siga o roteiro abaixo, que cobre todas as camadas do sistema — frontend, API e banco de dados.

1. **Autenticação Inicial:** Acesse `http://k8s.local`. Na tela de login, utilize as credenciais padrão criadas pelo mecanismo de inicialização de dados (*Data Seed*):
 - **E-mail:** `guest@email.com`
 - **Senha:** `guest1234`
2. **Cadastro de Participantes:** Navegue até a seção **Groups** e utilize o formulário de **Register** para cadastrar 3 ou 4 usuários fictícios. Isso valida a escrita no banco de dados via API.
3. **Criação de Grupo:** Crie um novo grupo (ex: "Viagem de Férias") e adicione os participantes recém-cadastrados. Salve o grupo.
4. **Acesso ao Dashboard:** Navegue até a aba **Dashboard** e selecione o grupo criado. O painel deve exibir os saldos zerados e o histórico de transações vazio.
5. **Inclusão de Despesas:** Adicione uma nova transação (ex: "Jantar" · R\$ 300,00). Selecione quem pagou e clique em **Split Equally** para dividir igualmente entre os participantes selecionados. Salve a transação.
6. **Validação do Algoritmo:** Adicione mais uma ou duas despesas alternando o pagador. O Dashboard deve atualizar os saldos líquidos e exibir, na seção de liquidação, o resultado do algoritmo de *Priority Queues* com o número mínimo de transações para zerar as dívidas do grupo.

Nota de limpeza de ambiente: Para remover completamente a instalação e recriar o ambiente do zero (inclusive apagando os dados do banco), execute:

```
helm uninstall social-ledger
kubectl delete pvc social-ledger-db-pvc
```

7. Comandos Úteis

- Verificar status dos pods: `kubectl get pods`
- Ver logs da API em tempo real: `kubectl logs -f deployment/social-ledger-api`
- Ver logs do frontend: `kubectl logs -f deployment/social-ledger-frontend`
- Inspecionar o Ingress: `kubectl describe ingress social-ledger-ingress`
- Verificar os valores do Helm chart: `helm get values social-ledger`
- Remover a instalação (mantendo PVC): `helm uninstall social-ledger`
- Remover tudo inclusive dados: `helm uninstall social-ledger && kubectl delete pvc social-ledger-db-pvc`
- Renderizar templates sem instalar: `helm template social-ledger ./helm/social-ledger`